

The f1r3score Player

Specification for a Rust tool that reads a textual score and plays a run of it

F1R3FLY.io

Specification — 26 September 2026

Revision 2. Implements *Scores as Processes* (second version, as revised in **publications**
commit 5740fe9)

Abstract

This specification defines **f1r3score**, a Rust command-line player and library for the score calculus of *Scores as Processes*. The player reads a score written in a textual surface syntax, elaborates it to a closed term of the calculus, and executes one run. At each location it matches receipts and messages on their subjects alone, with either decoration component possibly left open; it evaluates the graded where-clauses on the candidates, groups them into contention sets, selects a maximal matching with probability proportional to the product of its weights, and keeps the *record* of every communication. Playback reads each note off its record. The outputs are the performance (the multiset of timed notes of positive length), a replayable trace of records, a Standard MIDI File, and optionally live MIDI.

Three properties organise the design. *Faithfulness*: the engine implements the matching, reduction and resolution rules of the note literally, with no knowledge of voices, keys, keyboards, players or idioms; those are library definitions in the surface syntax. *Exactness*: weights are exact rationals and chance is consumed by exact interval decoding, so a score, a chance source and a scheduler determine a run bit for bit on every platform. *Conformance*: the reference simulator **skeinsim.py** and its fifteen tests, T0–T14, are ported as the acceptance suite with the note’s thresholds.

Revision 2 follows the revised note: wildcard decorations and the matching relation, unlabelled reduction with records and playback, the key/touch vocabulary, and the new section on musicians and instruments (self-re-arming keys, the $2n$ -hands piano, synchronous output, the chimeric instrument). §1.4 lists what changed.

Contents

1	Purpose and scope	3
1.1	What the tool does	3
1.2	What was reviewed	3
1.3	What the tool is not	3
1.4	What changed in revision 2	4
1.5	Conventions	4
2	The semantic contract	4
3	The textual format	5
3.1	Design position	5
3.2	Declarations	6
3.3	Core process syntax	6

3.4	Definitions: the elaboration-time macro layer	6
3.5	The standard library	7
3.6	The instrument layer	8
3.7	Twinkle, in full	9
3.8	Diagnostics and lints	9
4	Architecture and data model	9
4.1	Crates	9
4.2	Terms and hash-consing	10
4.3	The soup	10
4.4	Digests	11
5	The clause language	11
5.1	Grammar	11
5.2	Atoms and the candidate view	12
5.3	Tables and machines	12
5.4	Evaluation	13
5.5	The checkable fragment	13
6	The engine	14
6.1	One resolution	14
6.2	Maximal matchings	14
6.3	Scheduling simultaneous sets	15
6.4	Time	15
6.5	Termination and productivity	15
6.6	Collection of inert locations	15
7	Chance sources	16
7.1	One interface	16
7.2	Which source a choice uses	17
7.3	Factorised resolution	17
7.4	Human choice	17
8	Outputs	18
8.1	Playback	18
8.2	Performance	18
8.3	Trace	18
8.4	MIDI	18
8.5	Live MIDI	19
9	Command line	19
10	Conformance	19
10.1	The ported suite	19
10.2	Additional tests	21
11	Phasing	22
12	Decisions to confirm	22

1 Purpose and scope

1.1 What the tool does

Given a file `piece.score`, the command

```
f1r3score play piece.score --chance prng:7 --notes 64 --midi piece.mid
```

parses and elaborates the score, runs it until 64 notes of positive length have sounded or nothing more can fire, writes the performance to standard output, and writes a MIDI file. The same run, replayed from its trace, reproduces the same performance exactly; and because a trace holds records, it can be played back without re-running the score at all.

1.2 What was reviewed

- `publications/f1r3score/score-calculus.tex`, second version as of commit 5740fe9 (“removed labelling scheme on reduction rules; added discussion of player versus instrument”): the normative semantics. Numbers of definitions, results and sections below refer to that text.
- `publications/f1r3score/skeinsim.py` (853 lines, tests T0–T14): the reference simulator. Its data model — a soup indexed by location alone, each item keeping its own subject timbre, hash-consed quotes, generic candidate generation, playback from records — is the model this specification adopts.
- `publications/fuzzyware/graded-where-pln.tex` (Paper II): the two formula sorts, per-candidate evaluation, contention sets, and the resolution algebra $(\mathbb{R}_{\geq 0}, +, \times, 0, 1)$.
- `publications/f1r3skein/f1r3skein-drawn-distributions.tex`: drawn weighted state machines, their canonical form, the `Machines` Theory, and exact interval decoding (Algorithm 1).
- `F1R3Games/F1R3Skein`: the Rust workspace (`spigot_stream`, `dual_spigot`, `spigot_midi`, `leap_spigot`), for reusable digit streams and MIDI code.
- The rho-weighted crate and its design (`publications/rho-life/Gillespie`), for the ideal-versus-checkable discipline expressed as types.

1.3 What the tool is not

- **Not an interpreter on a shard.** Following the precedent set for the weighted-GSLT simulator, the player is a separate, offline execution path, with no dependency on `f1r3node-rust`, `RSpace` or consensus.
- **Not F1R3Skein.** The score calculus generalises an idea from the instrument; the instrument’s streams are one score among many (Remark 6.3). A drawn machine can be imported as a clause factor, nothing more.
- **Out of scope:** tempo as a graded dimension, dynamics and articulation as decorations, orchestration as grading (a fan over timbres with the carried timbre in the view, open thread 4), complex grading, and learning styles from a corpus. The renderer takes a fixed tempo as a rendering parameter only.

1.4 What changed in revision 2

change in the note	consequence here
Decorations may be the wildcard .; @P is the general name $\langle @P, \rightarrow, \rightarrow \rangle$ (Def. 2.1)	Names carry optional timbre and datum; surface syntax gains $\rightarrow$ in decorations, @P and $\mathbf{x}!(Q)$ shorthands (§3).
Matching on subjects only; timbres meet; payload takes no part (Defs. 2.4, 2.7)	The payload-timbre check of revision 1 is gone. The soup is indexed by location alone, and one contention set may span several timbres (R2.4, §6).
Round trip $x \sqsubseteq @*x$ as an instance relation, not an equation (Prop. 2.5, Rem. 2.6)	Congruence and name equivalence unchanged; instance order used only by matching; property test added.
Reduction unlabelled; the note is read off the record (Def. 2.8)	The engine emits records; playback is a separate pure function; traces store records; a new render command plays a trace back (R2.5, §8).
“Hand” becomes <i>key</i> ; a co-channel message is a <i>touch</i> Requests, refreshes and written notes pass general names	Vocabulary followed throughout. Revision 1’s “stream keys” are renamed <i>stream labels</i> to avoid the clash. std.score updated; T10 checks every request is a general name.
Hocket across timbres allowed (Step ^{ψ, τ} , §7.4)	std.score hocket takes two timbres.
New §9: keys that re-arm, keyboards, 2n-hands piano, synchronous output, chimera	Library additions (§3.6); ; sugar; reserved control timbre; T13, T14; collection rule tightened (Finding 6.5).

1.5 Conventions

MUST, SHOULD and MAY have their RFC 2119 meanings. Requirements are numbered Rn and decisions Dn. Rust identifiers are indicative; the phase exit criteria are normative.

2 The semantic contract

This section restates, as obligations on the implementation, what the note defines. Anything not stated here follows the note.

Requirement 2.1 (Data). A score fixes a finite pitch alphabet Π containing the rest $\mathbf{r}$, a finite duration alphabet Δ with lengths $|\cdot| : \Delta \rightarrow \mathbb{Q}_{\geq 0}$ in whole notes, containing the null duration ϵ with $|\epsilon| = 0$, and a finite set of timbres. Π and Δ MUST be disjoint, since a concrete datum fixes the polarity of its name. Two timbres are reserved and never rendered: the dead timbre $\tau_{\perp}$, used by bases and key locations, and the control timbre κ , used by synchronous output (Def. 9.3).

Requirement 2.2 (Terms). A decoration component is concrete or the wildcard: $\hat{\tau} ::= \tau \mid _$ and $\hat{d} ::= d \mid _$. Names are variables or triples $\langle @P, \hat{\tau}, \hat{d} \rangle$; processes are 0, receipts $\mathbf{for}(y \leftarrow x \text{ where } \psi)P$, joins (chord receipts, Def. 6.5), messages $\mathbf{x}!(Q, \hat{\tau}, \hat{d})$, parallel composition and $*x$. There is no replication, no **new**, and no recursion former. A name whose datum is $_$ has no polarity.

Requirement 2.3 (Congruence and equivalence). Structural congruence is the least congruence closed under α -conversion making $(\mid, 0)$ a commutative monoid and satisfying $*\langle @P, \hat{\tau}, \hat{d} \rangle \equiv P$. Two names are equivalent when their quoted processes are congruent and their decorations are equal, a wildcard being equal only to a wildcard (Def. 2.3). The implementation MUST decide name equivalence exactly, and MUST *not* identify $@*x$ with x (Rem. 2.6): the round trip is the instance relation $x \sqsubseteq @*x$, used by matching and by nothing else.

Requirement 2.4 (Matching and communication). A candidate is a top-level receipt and a top-level message whose subjects *match* (Defs. 2.4 and 2.7): their locations are equivalent; their timbres meet in a concrete timbre, $\hat{\tau}_1 \sqcap \hat{\tau}_2 = \tau$; and their data are both concrete and of opposite polarity. The payload $(Q, \hat{\tau}_3, \hat{d})$ takes no part in matching and is carried across unchanged: firing continues with $P\{\langle @Q, \hat{\tau}_3, \hat{d} \rangle / y\}$. In particular two subjects whose timbres are both $-$ never communicate, nor does a subject with no polarity (Prop. 2.10(ii)).

Requirement 2.5 (Records and playback). Reduction is unlabelled. Each firing yields a *record*: the receipt’s subject $\langle @S, \hat{\tau}_1, d_1 \rangle$, the message’s subject $\langle @T, \hat{\tau}_2, d_2 \rangle$, and the payload $(Q, \hat{\tau}_3, \hat{d})$ (Def. 2.8). Its note is $\nu = (\hat{\tau}_1 \sqcap \hat{\tau}_2, \pi, \delta)$, with π whichever datum is a pitch and δ whichever is a duration. The engine **MUST** emit records and **MUST** compute notes only through a single pure function ν on records, the *playback* function; nothing in the engine maintains a separate label. Every record has a note; there are no silent steps.

Requirement 2.6 (Weights and resolution). A candidate at a receipt with clause ψ is *available* iff $\psi \llbracket c \rrbracket > 0$. Contention sets are the connected components of the relation “shares a receipt or a message” over available candidates. A set is resolved by choosing a maximal matching $\mathcal{M}$ with probability proportional to $\prod_{c \in \mathcal{M}} \psi_c \llbracket c \rrbracket$ and firing all of it (Def. 3.3).

Requirement 2.7 (Time). Every top-level receipt and message carries a stamp in $\mathbb{Q}_{\geq 0}$. A communication has onset $\max(t_r, t_m)$; its timed note is (t, τ, π, δ) where (τ, π, δ) is the note of its record; it stamps what its continuation brings to top level with $t + |\delta|$ (a chord, with $t + \max_i |\delta_i|$). Runs are onset-ordered: a set is resolved only when its onset, the least onset of its candidates, is minimal among all sets. The performance is the multiset of timed notes of positive length (Def. 2.11).

Requirement 2.8 (Engine neutrality). The engine **MUST** know nothing of voices, fans, keys, servers, keyboards, players, synchronous output, chimeras, machines or idioms. Those are definitions in the surface syntax that elaborate to terms. (This is the property `skeinsim.py` states in its header, and it is what makes Finding 10.3 — the reflective voice plays the notes of the step semantics — and Findings 10.8 and 10.9 on instruments tests of the calculus rather than of the code.)

Remark 2.9 (What the note leaves to the implementation). Three things are parameters of the semantics rather than parts of it: the chance source (Def. 3.3 names it a parameter, and it may be a person), the order in which simultaneously due contention sets are resolved, and whether a set fires a maximal matching or a single candidate. The last is `mode='one'` in the simulator and is retained only as a diagnostic (T3). The first two are explicit and recorded in every trace (§7, §6.3).

3 The textual format

3.1 Design position

Decision 3.1 (A dialect of the flr3—@ng control flow sublanguage). The surface syntax reuses the concrete syntax of the flr3—@ng CFL wherever the calculus has the same former: `for (y <- x where ...) {P }, x!(...), |, *x, @P`. The revised note makes the fit closer: its general name $@P$ is exactly the rholang name $@P$, and a message that leaves its payload decoration open is exactly the rholang `send x!(Q)`. What is new is the decoration of names, the alphabet declarations, and a small elaboration-time macro layer. The grammar is kept small enough to be restated as a **Theory Score** in the flr3—@ng DDL once the Theory syntax lands in the node; the parser crate is a replaceable front end over a stable core AST (§4).

Files use the extension `.score` and are UTF-8. Comments are `//` to end of line and `/* ... */`.

3.2 Declarations

A score file is a sequence of imports, declarations and definitions, followed by exactly one **play** item giving the initial configuration.

```
score Twinkle

pitches    { r, C3, F3, G3, C4, D4, E4, F4, G4, A4 } // r: the rest, required
durations  { q = 1/4, h = 1/2 } // eps = 0 is implicit
timbres    { piano = gm(0) on 1, bass = gm(32) on 2 } // GM program, MIDI channel
```

- Requirement 3.2** (Alphabets). (i) **pitches** lists Π *in order*; that order defines **step**(k) (§5.2). **r** MUST be present and is unordered. A pitch written in scientific notation (C4, F#3, Bb2) gets its MIDI number implicitly (C4 = 60); any other identifier MUST be given one, $\text{deg7} = 67$.
- (ii) **durations** gives each length as a non-negative rational literal (1/6 for a triplet eighth, 3/8 for a dotted quarter). **eps** = 0 is always present and MUST not be redeclared.
- (iii) Ranges MAY abbreviate alphabets: **pitches** {r, scale major C4 .. C7 }, **pitches** {r, piano88 } (A0 to C8). Expansion is at elaboration time; the alphabet is always finite.
- (iv) Identifiers in Π , Δ and the timbre set MUST be pairwise distinct, and MUST differ from $_$, **dead** and **ctl**, the surface names of $\tau_{\perp}$ and κ .

3.3 Core process syntax

calculus	surface	note
0	0	
$\langle @P, \hat{\tau}, \hat{d} \rangle$	$\langle @P, \text{tau}, d \rangle$	$_$ for an open component
$@P = \langle @P, _ , _ \rangle$	@P	the general name
$x!(Q, \hat{\tau}, \hat{d})$	$x!(Q, \text{tau}, d)$	
$x!(Q, _ , _)$	$x!(Q)$	passes the general name @Q
$\text{for}(y \leftarrow x \text{ where } \psi)P$	$\text{for } (y \leftarrow x \text{ where } \text{psi}) \{P\}$	clause optional
join	$\text{for } (y1 \leftarrow x1 \ \& \ y2 \leftarrow x2 \text{ where } \text{psi}) \{P\}$	chord
$P \mid Q$	$P \mid Q$	
$*x$	$*x$	
stamp	$\text{at } 3/4 \{P\}$	top level of play only

Compound quoted processes are parenthesised: $\langle @P \mid Q \rangle$, **piano**, **q**>. In binder position $_$ is a binder that is not used; in decoration position it is the wildcard; the grammar keeps the two positions apart. In the **play** item every top-level receipt and message is stamped 0 unless an **at** says otherwise (an entrance).

3.4 Definitions: the elaboration-time macro layer

The note builds everything from abbreviations: **N**, **Rec**, **Fan**, **Step**, **Server**, **Voice**, **Call**, **Resp**, and now **Key**, **Installed**, synchronous output and **Chimera**. The surface syntax needs exactly this facility and no more.

```
def N(L: proc, tau: timbre, p: pitch, d: dur; K: proc) =
  for ( _ <- <@L, tau, p> ) { K } | <@L, tau, d>!(0) // passes @0 (Section 2.5)
```

- Requirement 3.3** (Definitions are macros). (i) Parameters are sorted: **proc**, **name**, **timbre**, **pitch**, **dur**, **clause**, **string**, **pitchset** (a literal subset such as {A2, E2} or

a declared set), finite lists and tuples of these, and **cont** (a definition of arity one in a name, passed by name). Timbre and datum arguments may be `_`. A definition may return a name (**def touch(...): name = ...**).

- (ii) Definitions are expanded at elaboration time and **MUST** form an acyclic call graph. A recursive definition is rejected with a diagnostic pointing to §5.2 of the note: unbounded behaviour is obtained by reflection, not by recursion (Rem. 2.2).
- (iii) Expansion is capture-avoiding. The elaborated term is a closed term of the core syntax and nothing else.

Finite products. $\prod_{v \in \Delta^+} \prod_{t \in \Pi}$ is written as a comprehension that expands to a parallel composition:

```
def Fan(x: name, tau: timbre, over: pitchset) =
  par v in durations+, t in over {
    <@*x, tau, v>!( <@*x, tau, v>(*x, tau, t), tau, t )
  }
```

Generators range over **pitches**, **pitches-r** (the ordered pitches), **durations**, **durations+**, declared pitch sets, and explicit lists.

Remark 3.4 (Fans over a subset). A voice's fan offers $|\Delta^+| \cdot |\Pi|$ messages per note. With an 88-key alphabet, needed for a piano, that is 440 per note for five durations. A fan over a declared pitch set $\Pi' \subseteq \Pi$ is the same score as the full fan with every pitch outside Π' given weight 0 by a crisp factor; it is offered in **std.score** as an optimisation with that meaning, and a test **MUST** check the equivalence.

Lines. A written part is a right-nested chain of written notes. The derived form

```
line(Lm, piano) [ C4 q, C4 q, G4 q, G4 q, A4 q, A4 q, G4 h ]
```

elaborates to $N_{Lm, \text{piano}}(C4, q; \dots N_{Lm, \text{piano}}(G4, h; 0) \dots)$. An optional trailing **then** *K* replaces the final 0 with a continuation, which is how a written phrase hands off to a responder (§7.4 of the note).

Synchronous output. Definition 9.3's sequencing is surface syntax:

```
touch(P1, A3, q)!(0) ; touch(P1, C3, q)!(0) ; ( touch(P1, A3, h)!(0) | touch(P1, E3, h)!(0) ) ; 0
```

A step is one touch or a parenthesised parallel group of touches; `;` associates to the right and binds more tightly than `|`, so `a ; b ; 0 | c ; 0` is two players in parallel. Each `;` elaborates to the payload-augmented touches and Wait_A^m of Def. 9.3, with a location *A* fresh for that elaboration site (§12, item 4) and the control timbre **ctl**.

Locations. A *base* is written **base** "Lm" and elaborates to the closed receipt $\text{for}(_ \leftarrow \langle @0, \tau_\perp, r \rangle) 0$ tagged with the string; distinct tags give inequivalent bases. A *key location* is written **keyloc**(*P*, *k*) and elaborates to $K_{P,k} = \langle @B_P, \tau_\perp, k \rangle!(0, _, _)$ (§9.1), B_P being the keyboard's base. User code **MUST** not write **dead**; only these two builtins emit it.

3.5 The standard library

Requirement 3.5 (**std.score**). The distribution **MUST** ship **std.score**, written entirely in the surface syntax, defining: **N**, **line**; **Rec**, **Fan**, **Seed** (H_0); **Step**, **Req**, **Server**, **Refresh**, **Voice** (§5.2); **Call**, **Resp** (handoff, including the wildcard-timbre responder of §7.4); **Hocket** over two players with two timbres; the dual (rhythm-leads) **Fan** and **Step**; and the instrument layer of

§3.6. **Voice** MUST be the reflective voice, with no engine support: its server keeps its own code on a code channel and re-sends it (Prop. 5.3), and every request passes the general name of the new location.

```
// std.score, excerpt: the reflective voice
def Req(z: name, tau: timbre, BS: proc) = <@BS, tau, eps>!(*z)      // passes @*z

def Server(tau: timbre, psi: clause, BS: proc, BC: proc) =
  for (w <- <@BS, tau, r>) {
    Step(w, tau, psi, Req(_, tau, BS)) | Refresh(tau, BC)
  }

def Refresh(tau: timbre, BC: proc) =
  for (c <- <@BC, tau, r>) { *c | <@BC, tau, eps>!(*c) }

def Voice(tau: timbre, psi: clause, p0: pitch, u0: dur, tag: string) =
  Server(tau, psi, base tag ++ "/S", base tag ++ "/C")
  | <@(base tag ++ "/C"), tau, eps>(Server(tau, psi, base tag ++ "/S",
                                          base tag ++ "/C"))
  | <@(base tag ++ "/S"), tau, eps>(Seed(base tag, tau, p0, u0))
```

3.6 The instrument layer

```
// std.score, excerpt: keys that re-arm themselves (Definition 9.1)
def Key(P: proc, k: pitch, tau: timbre, psi: clause) =
  for (y <- <@keyloc(P, k), tau, k> where psi) { *y | KeyRefresh(P, k, tau) }

def KeyRefresh(P: proc, k: pitch, tau: timbre) =
  for (c <- <@keycode(P, k, tau), tau, r>) { *c | <@keycode(P, k, tau), tau, eps>!(*c) }

def Installed(P: proc, k: pitch, tau: timbre, psi: clause) =
  Key(P, k, tau, psi) | <@keycode(P, k, tau), tau, eps>(Key(P, k, tau, psi))

def Keyboard(P: proc, tau: timbre, psi: clause, over: pitchset) =
  par k in over { Installed(P, k, tau, psi) }

def Chimera(P: proc, k: pitch, voices: [(timbre, clause)]) =
  par (tau, psi) in voices { Installed(P, k, tau, psi) }

def touch(P: proc, k: pitch, d: dur): name = <@keyloc(P, k), _, d>  // timbre left
open
```

`keycode(P, k, tau)` is a builtin like `keyloc`, giving the key's code location $C_{P,k,\tau}$. The paper's player notation $\langle @P, k, d \rangle$ puts a key where a timbre usually stands; the surface syntax does not copy that overloading and writes `touch(P, k, d)` instead. A touch that names a timbre is `<@keyloc(P, k), vibes, d>`.

```
score FourHands                                     // Example 9.5; golden score for T13
import std
pitches    { r, A2, E2, C3, E3, A3 }
durations  { q = 1/4, h = 1/2 }
timbres    { piano = gm(0) on 1 }

def P1 = base "P1"   def P2 = base "P2"

play Keyboard(P1, piano, 1, {A2, E2, C3, E3, A3})
```

```

| Keyboard(P2, piano, 1, {A2, E2, C3, E3, A3})
| touch(P1, A3, q)!(0) ; touch(P1, C3, q)!(0) ; touch(P1, E3, q)!(0) ;
  ( touch(P1, A3, h)!(0) | touch(P1, C3, h)!(0) | touch(P1, E3, h)!(0) ) ; 0
| touch(P2, A2, h)!(0) ; touch(P2, E2, h)!(0) ; 0

```

3.7 Twinkle, in full

```

score Twinkle
import std
pitches { r, C3, F3, G3, C4, D4, E4, F4, G4, A4 }
durations { q = 1/4, h = 1/2 }
timbres { piano = gm(0) on 1, bass = gm(32) on 2 }

play line(base "Lm", piano) [ C4 q, C4 q, G4 q, G4 q, A4 q, A4 q, G4 h,
                             F4 q, F4 q, E4 q, E4 q, D4 q, D4 q, C4 h ]
  | line(base "Lb", bass) [ C3 h, C3 h, F3 h, C3 h, F3 h, C3 h, G3 h, C3 h ]

```

Requirement 3.6 (Golden scores). Twinkle MUST play, under every scheduler, the 22 timed notes of the table in §2.5, ending at 4 whole notes (T12). FourHands MUST play, under every scheduler, the performance of Example 9.5 with 16 null notes (T13).

3.8 Diagnostics and lints

The elaborator MUST reject: overlapping alphabets; undeclared data or timbres; recursive definitions; ill-sorted arguments; **dead** in user code; clauses outside the checkable fragment (§5.5). It SHOULD warn, citing Prop. 2.10(ii), when a receipt’s or a message’s subject has datum `_` (it has no polarity and can never communicate), and when a receipt and a message at one location both leave the timbre open with nothing else there to meet them. Revision 1’s warning about a payload timbre that differs from its subject’s is withdrawn: under the revised rules that is how a line passes between instruments.

4 Architecture and data model

4.1 Crates

crate	responsibility
score-core	data alphabets, decorations with wildcards, hash-consed terms, congruence normal form, substitution, names, locations, matching and the instance order. No I/O.
score-syntax	lexer, parser, surface AST, macro elaboration, std.score , pretty-printer, diagnostics with spans.
score-logic	crisp and graded formula ASTs, the checkable-fragment type, candidate views, evaluation, tables and machines.
score-engine	the soup, candidate generation, contention sets, maximal matchings, resolution, time, scheduling, collection, records.
score-chance	chance sources: PRNG digit streams, spigot digit streams, exact interval decoding, human choice, replay.
score-render	playback (ν on records), performance and trace serialisation, Standard MIDI File writer, live MIDI (feature live).
f1r3score	the binary: command line, wiring, exit codes.
score-conformance	the ported skeinsim suite and golden files.

Requirement 4.1 (Portability). `score-core`, `score-syntax`, `score-logic`, `score-engine` and `score-chance` (without the `spigot` feature) MUST build for `wasm32-unknown-unknown` and MUST perform no I/O, so that the same engine can be embedded in a browser page or an Apple Vision Pro client. They SHOULD have no dependencies beyond `num-bigint` / `num-rational`. The workspace targets stable Rust, edition 2021, with `rust-version` pinned.

4.2 Terms and hash-consing

```
pub struct Datum(u16);           // index into Pi + Delta (disjoint)
pub struct Timbre(u16);          // includes the reserved DEAD and CTL
pub enum Hat<T> { Is(T), Wild }  // a decoration component
pub enum Polarity { Channel, CoChannel } // defined only for Hat::Is(datum)
pub struct ProcId(u32);          // hash-consed, congruence-normal process

pub enum Name {
  Var(VarIdx),
  Quote { proc: ProcId, timbre: Hat<Timbre>, datum: Hat<Datum> },
}

pub enum Proc {                  // normal form only
  Nil,
  Par(Box<[ProcId]>),             // flattened, >= 2, sorted by id, no Nil
  Recv { subjects: Box<[Name]>, clause: ClauseId, body: ProcId },
  Send { subj: Name, payload: ProcId, ptimbre: Hat<Timbre>, carry: Hat<Datum> },
  Drop(Name),                    // on a variable only
}

impl Hat<Timbre> { pub fn meet(self, other: Self) -> Option<Self>; } // None: no
match
pub fn instance_of(x: &Name, general: &Name) -> bool;             // x below
general
```

Requirement 4.2 (Congruence by construction). The arena MUST intern only normal forms: parallel compositions flattened, 0 removed, children sorted by `ProcId` (a multiset), singletons unwrapped; $\langle @P, \hat{\tau}, \hat{d} \rangle$ replaced by P whatever the decoration; bound variables in de Bruijn form, so that α -equivalent terms are identical. Then $P \equiv Q$ iff their ids are equal, and location equivalence is integer equality. `Hat::Wild` is equal only to `Hat::Wild`. A property test MUST check interning against an independent, unoptimised congruence decision procedure on random terms with wildcards, and MUST check Proposition 2.5 on them: $x \sqsubseteq @*x$, $@*x$ is greatest at its location, and $@*x \equiv x$.

Remark 4.3 (Why sorting by id is sound). Children are interned before their parent, so sorting by id is a canonical order on already-normal children. Ids depend on interning order; they are never serialised as identities. Serialised terms and digests use a structural encoding (§4.4).

Requirement 4.4 (Substitution). Firing substitutes the name $\langle @Q, \hat{\tau}_3, \hat{d} \rangle$ built from the payload — decoration components possibly — for the bound variable and re-normalises. Substitution MUST be memoised on (body, name), since the same body is instantiated repeatedly by a voice and by every re-armed key.

4.3 The soup

The running configuration is not stored as a term. As in the simulator, it is a *soup*: the top-level receipts and messages, each an *occurrence* with a stable occurrence number and a stamp, indexed by *location alone*.

```

pub struct Loc(ProcId); // the quote; timbre is NOT part of the
  key
pub struct RecvOcc { id: OccId, subjects: Box<[Subject]>, clause: ClauseId,
  body: ProcId, stamp: Time, label: StreamLabel }
pub struct SendOcc { id: OccId, subj: Subject, payload: ProcId,
  ptimbre: Hat<Timbre>, carry: Hat<Datum>, stamp: Time }
pub struct Subject { loc: Loc, timbre: Hat<Timbre>, datum: Hat<Datum> }
pub struct Soup {
  recv: BTreeMap<Loc, Vec<RecvOcc>>,
  send: BTreeMap<Loc, Vec<SendOcc>>,
  dirty: BTreeSet<Loc>,
  next_occ: OccId,
}

```

Requirement 4.5 (Location-only index). The index key **MUST** be the location alone; each occurrence keeps its own subject timbre, which may be $\perp$. Indexing by (location, timbre), as revision 1 and the first simulator did, would miss every candidate in which one side leaves its timbre open — every touch on a keyboard, every chimera — and would split contention sets that the note makes one (Example 9.7).

Requirement 4.6 (Determinism of iteration). Every collection whose iteration order can influence a run **MUST** be ordered (**BTreeMap**, sorted **Vec**), keyed by occurrence number or by a structural key. Occurrence numbers are assigned in firing order. Consequently a score, a chance source and a scheduler determine the run exactly.

Spawning a process at stamp t decomposes its normal form: **Par** spawns each child; **Recv** and **Send** become occurrences at their subjects' locations; **Nil** does nothing. A top-level $*y$ cannot occur in a closed term.

4.4 Digests

Every elaborated score has a *digest*: SHA-256 over a canonical, id-independent serialisation of its term, alphabets and lengths. The digest heads every trace and performance file and is the identity under which a score would later be stored as a content-addressed value on a shard.

5 The clause language

5.1 Grammar

The two sorts of Paper II, with the atoms of Definition 3.1 of the note.

sort	surface	meaning
graded ψ	$3/2, 0.3$	constant $w \in \mathbb{Q}_{\geq 0}$
	$[\text{beta}]$	$[\beta]$
	$\text{psi} * \text{psi}, \text{psi} + \text{psi}$	$\otimes, \oplus$
	$\langle \rangle \text{psi}$	graded behavioural modality $\langle K \rangle_{\mathbf{v}}$
	$\mu X. \text{psi}, \nu X. \text{psi}$	graded fixed points
	$\text{table key } \{ \dots \}, \text{machine } \dots$	sugar, below
crisp β	factor name	reference to a named factor
	$\text{true}, !b, b \ \&\& \ b, b \ \ b$	propositional
	atoms	§5.2
spatial σ	$\langle \rangle b, \text{leads}(b), \nu X. b$	behavioural
	$\text{true}, 0, s \ \ s, !s, s \ \&\& \ s$	over processes
	$\langle n, \text{tau}, d \rangle ! (s, \text{tau}, d)$	out: a message
	$\text{for}(\langle n, \text{tau}, d \rangle) s$	in: a receipt
	$@s$	name predicate

In message and receipt patterns, `?` matches any location, timbre or datum, *including* a wildcard, while `_` matches only a wildcard component: a formula can distinguish a general name from a decorated one. `n` is itself a spatial formula applied to the quoted location, so patterns nest. Graded formulae have no negation (Paper II, Definition 2.4).

5.2 Atoms and the candidate view

A clause is evaluated on a candidate c against the configuration in which it stands. The view is

```
pub struct View<'a> {
  pub timbre: Timbre,           // the meet: always concrete
  pub pitch: Datum, pub dur: Datum, // the note nu(record(c))
  pub carry: Hat<Datum>,       // kappa(c); may be open
  pub ptimbre: Hat<Timbre>,    // the payload's timbre; may be open
  pub loc: ProcId,             // @S
  pub passes: ProcId,          // Q of <@Q, ptimbre, carry>
  pub chord: &'a [ChordSlot],  // joins only
  pub residue: Lazy<'a, Residue>, // behavioural atoms only
}
```

The view is built from the candidate's prospective record by the same playback function ν that renders performances (R2.5), so what a clause sees as the note and what is heard cannot diverge.

atom	holds when
<code>pitch(s), dur(v)</code>	the note's pitch, its duration equal the argument
<code>carry(e)</code>	the carried datum is concrete and equals e ; <code>carry(_)</code> holds when it is open
<code>step(k)</code>	carry and pitch are concrete ordered pitches and $\text{ord}(\kappa) - \text{ord}(\pi) = k$
<code>at(s)</code>	the location's quote satisfies σ
<code>passes(s)</code>	the passed process satisfies σ
<code>held(t)</code>	<code>at(<?, ?, ?>!(true, ?, t))</code> (derived)
<code>prev(u)</code>	<code>at(<?, ?, u>!true)</code> (derived)
<code>back(j, e)</code>	the record j levels into the past carries e (derived)
<code>last[m1 .. mk]</code>	<code>at(Last(m))</code> , §7.1 (derived)
<code>pitch#i, dur#i, carry#i</code>	per-pattern atoms of a chord receipt
<code>leads(b)</code>	the residue of firing c , in the present configuration, satisfies β

Remark 5.1 (The note's timbre and the payload's timbre are not atoms). Definition 3.1 exposes the note through `pitch` and `dur` only; a clause sits on a receipt whose own timbre is usually fixed, as each key of a chimera is. The view carries `timbre` and `ptimbre` for playback and for the reserved orchestration extension (open thread 4), but no atom reads them in this version. Adding `timbre(t)` and `ctimbre(t)` is the whole of the clause-side work that open thread would need.

Requirement 5.2 (Derived atoms are sugar). `held`, `prev`, `back`, `last` MUST elaborate to spatial formulae. The evaluator MAY compile them to direct walks over the record chain (as the simulator's `back` and `repeat` do), but a test MUST check the compiled and the formula forms agree on random views, exactly as T9 does for tables.

5.3 Tables and machines

`table key {k: w, ..., _: w }` is a sum over mutually exclusive crisp conjunctions evaluated by lookup, keyed by `dur`, `(pitch, carry)`, `(prev, dur)`, `step`, or any tuple of atoms

that are functions of the view. `_` is the default (absent means 0); ranges such as `-2..2` are allowed on `step`. The `dur` key is what a chimera’s articulation clauses use (Example 9.7):

```
factor vibes_temper    = table dur { e: 9, q: 1, h: 1 }
factor strings_temper = table dur { e: 1, q: 1, h: 9 }
```

`machine pitch {s -> t @ w, ... }` and `machine dur {u -> v @ w, ... }` are Example 6.2’s factors: tables keyed by `(pitch, carry)` and `(prev, dur)`. The arrow syntax is that of the **Machines** Theory in the drawn-distributions design; `machine pitch import "file.machine"` reads a canonical machine exported by the instrument, mapping its state indices onto the declared pitch alphabet by position (the rest as the last state). `machine dur uniform` is the complete uniform machine.

5.4 Evaluation

Requirement 5.3 (Exact weights). Clause values are exact non-negative rationals (`BigRational`, with a fast path for `u64/u64` that promotes on overflow). Decimal literals are read exactly (`0.3` is `3/10`). No floating point enters a weight, a normalisation, or a choice.

Requirement 5.4 (Purity). Evaluation is a pure function of the view and the configuration. Evaluating a clause on a candidate that does not fire changes nothing.

$\otimes$ short-circuits on 0; $\oplus$ sums; $\lceil \beta \rceil$ is 1 or 0. Graded fixed points are evaluated only inside the checkable fragment, by iteration to the exact fixed point over a finite state space; if the iteration does not reach an exact fixed point within the declared bound the score is rejected at elaboration, never approximated at run time.

5.5 The checkable fragment

The ideal logic defines; the fragment decides. As in **rho-weighted**, the distinction is a type, not a comment.

```
pub struct Checked<F> { formula: F, class: Locality, depth: usize }
pub enum Locality { Local, Behavioural { window: usize } }
impl<F> Checked<F> { pub fn try_new(f: F, alph: &Alphabets) -> Result<Self, WhyNot>;
}
```

Requirement 5.5 (Fragment membership). A clause is admitted iff it is in one of two classes.

- (i) **Local**: no behavioural modality, no `leads`. Its value depends only on the view. Spatial formulae are decidable on the finite quoted terms involved; separating conjunction is decided by enumerating splits of a multiset and `MUST` be bounded by a configured maximum split count.
- (ii) **Window-behavioural**: behavioural modalities and fixed points are admitted when every spatial formula in *every* clause of the score reads the past to depth at most d and the alphabets are finite. Then the state relevant to the formula is finite (§7.2), and ν is computed exactly by Knaster–Tarski iteration over the abstract graph whose nodes are configurations with every location truncated to depth d and stamps erased.

Anything else is rejected with `WhyNot` naming the offending subformula.

Decision 5.6 (Behavioural modalities read the actual company). In Proposition 7.2 the contexts K range over “the one-hole contexts supplied by the rest of the score”. A player has exactly one such context at each moment: the rest of the current configuration. The player

evaluates $\langle K \rangle$ and **leads** against that, and the $\langle \rangle$ surface form carries no explicit context. This is consistent with Remark 2.9: in an open system the context label supplies the missing half of a record, and here the missing half is always present in the soup. Quantifying over hypothetical companies is an analysis task, not a playing task, and is out of scope.

Requirement 5.7 (Locality classes drive caching). Local clauses depend only on the candidate, so a location’s candidate list is recomputed only when that location is dirty. If any receipt in the soup carries a behavioural clause, every such receipt’s candidates are re-evaluated after every resolution. The trace header records which regime the run used.

6 The engine

6.1 One resolution

1. **Candidates.** For each dirty location, pair every receipt with every message there whose subject matches it (R2.4): timbres meet concretely, data concrete and of opposite polarity. For a join, take every injective assignment of matching messages to its patterns, all at that location. Build the prospective record, the view from it, evaluate the clause, keep the candidates of positive weight.
2. **Contention sets.** Union-find over shared receipt or message occurrence numbers, per location. Communication requires equivalent locations, so a set never spans two locations (the engine **MUST** assert this); it may span several timbres, as at a chimeric key touched with the timbre open.
3. **Onsets.** A candidate’s onset is $\max(t_r, t_m)$ (for a join, the maximum over all its stamps); a set’s onset is the least of its candidates’.
4. **Due sets.** Let $t_{\min}$ be the least onset over all sets. The due sets are those with onset $t_{\min}$; the scheduler picks one (§6.3).
5. **Alternatives.** Enumerate the maximal matchings of the chosen set in canonical order, each with weight $\prod_{c \in \mathcal{M}} \psi[c]$.
6. **Choice.** Ask the set’s chance source for an index with probability proportional to the weights (§7).
7. **Firing.** For each candidate in the matching: remove both occurrences, emit its record with its onset, form the continuation $P\{\langle @Q, \hat{\tau}_3, \hat{d} \rangle / y\}$ and spawn it at the new stamp. Mark affected locations dirty. Append one trace event.

Requirement 6.1 (Canonical order of alternatives). Candidates are ordered by (receipt occurrence number, message occurrence numbers); matchings by the lexicographic order of their sorted candidate lists. Together with R4.6, this makes “the i -th alternative” a well-defined, replayable quantity.

6.2 Maximal matchings

Requirement 6.2 (Stars and the general case). (i) **Star fast path.** If every candidate in a set shares one receipt, or every candidate shares one message, the maximal matchings are the single candidates. This covers every voice, every chord receipt, every null note, and every open-timbre touch at a chimeric key (one message, several keys), and **MUST** be $O(\text{set size})$ with no enumeration.

- (ii) **General case.** Otherwise enumerate maximal matchings of the bipartite (for joins, hypergraph) contention structure by backtracking with maximality pruning. The number of alternatives **MUST** be bounded by a configurable limit (default 10^5); exceeding it is a run-time error naming the location, never a silent truncation. Several players reaching for one key at once (Example 9.5 with a third player) is a small general case.
- (iii) **Mode one.** A diagnostic mode, off by default, in which a set fires a single candidate chosen with probability proportional to its weight. It exists to reproduce T3’s second column and is recorded in the trace header.

6.3 Scheduling simultaneous sets

When several sets are due at the same onset, the note is indifferent for independent parts (Theorems 2.14 and 8.1) and the order matters only where parts contend. The player makes the order an explicit, recorded parameter.

Decision 6.3 (Schedulers). **canonical** (default): the due set whose location has the least structural key (the digest of its quote). **random:SEED**: a dedicated PRNG, separate from every chance source, shuffles the due sets. **by-timbre:A,B,...**: a stated priority by the timbre of each set’s least candidate’s note, for the Twinkle-style “melody first / bass first” tests. The scheduler never consumes digits from a musical chance source.

Scheduling is audible where parts contend: in Example 9.5 with a third player, whether the first player’s A3 sounds at 0 or waits until $\frac{1}{4}$, taking the rest of that phrase with it, is decided by which due set fires first. T13 therefore runs 20 random schedules and requires exactly two performances.

6.4 Time

Stamps are exact rationals. Continuations spawned by a firing receive the stamp of R2.7. A re-armed key is therefore available again at $t + |\delta|$ and a touch waiting for it has onset at least that (Prop. 9.2); nothing in the engine implements “a key sounds once at a time” — it follows from stamps and consumption. Joins use the “longest duration” convention of Def. 6.5; since the note lists this as open thread 5, the convention is a single function in the engine, not a scattered assumption.

6.5 Termination and productivity

A run stops at the first of: no available candidate anywhere (the score has ended or every part has fallen silent); a **--notes N** limit on notes of positive length; a **--until T** limit on onsets (whole notes); a **--max-steps** limit on resolutions. Installed keys that are never touched again hold receipts forever; a finished piece on an instrument ends because no touch remains, not because the soup is empty.

Requirement 6.4 (Unproductive loops). The note does not yet give a syntactic productivity check (open thread 6). The engine **MUST** bound the number of consecutive zero-length notes at one onset (default 10^5) and stop with an **unproductive** error naming the locations involved. A voice plays exactly two null notes per note (Prop. 5.3) and a synchronously written touch exactly two (Prop. 9.4), so the bound is never approached by well-formed library code.

6.6 Collection of inert locations

A voice leaves $|\Delta^+| \cdot |\Pi| - 1$ inert messages and $|\Pi| - 1$ inert keys behind at every note (Findings 10.1 and 10.3). Keeping them is correct but costs memory linear in the length of the piece.

Finding 6.5 (The simulator’s collection test is unsound with instruments). `skeinsim.py` collects a location whose quote is a message (`node(loc[0])[0] == 'send'`) once it has no candidate, justified by freshness. The revised note’s key locations are messages too: $K_{P,k} = \langle @B_P, \tau_\perp, k \rangle!(0, -, -)$. Between a strike and the refresh that re-arms it, a key location can have no candidate; collecting it then would discard waiting touches, and the refresh would spawn the new key at a location the collector believed dead. The simulator is safe only by accident of representation — it interns key locations as opaque (`'key', P, k`) nodes, not as messages — and its T13 and T14 do not turn collection on. The criterion cannot be carried over as written to an implementation that builds key locations as the note defines them.

Requirement 6.6 (Freshness collection). Collection is off by default. With `--gc freshness`, collection applies only to *record-shaped* locations: quotes of the form $\langle @H, \hat{\tau}, v \rangle!(H, \hat{\tau}', t)$, a message whose payload is the quote of its own subject’s location — the shape of `Rec` and of nothing in the instrument layer. After a resolution at a record-shaped location that then has no available candidate, its occurrences are dropped and its key added to a tombstone set. If anything is later spawned at a tombstoned location, the engine **MUST** stop with **freshness-violated**: the collection was unsound for this score. For scores built from `std.score` voices this cannot happen (Lemma 5.5), and key locations are never candidates for collection.

Remark 6.7 (Why not reachability). Collecting a location because no live term mentions it is not sound in this calculus: any process can quote any closed term it can build, and bases and key locations are built from nothing. Freshness is a theorem about how voices use locations; Proposition 9.6 is the opposite fact about keys (“nothing at a key is ever left unchosen for good”). The player relies on each only where it holds.

7 Chance sources

7.1 One interface

```
pub trait Chance {
  /// Choose an index into 'weights' (exact, all > 0) with probability
  /// proportional to weight. Returns the index and what was consumed.
  fn choose(&mut self, weights: &[Weight], ctx: &ChoiceCtx) -> (usize, Consumed);
  fn describe(&self) -> SourceSpec;           // recorded in the trace header
}
pub trait DigitStream { fn base(&self) -> u32; fn next(&mut self) -> u32; }
```

Decision 7.1 (Every automatic source is a digit stream). Every non-human source is a stream of digits in some base, and every choice is made by the exact interval decoding of the drawn-distributions design (Algorithm 1): cumulative weights are cleared to a common denominator, digits narrow a b -adic interval, and the choice is the bin that contains it. There is no floating point and no rejection sampling, so a given digit stream yields the same choices on every machine. $K_{\max}$ (default 64 digits per choice) and its deterministic fallback are part of the source’s specification.

spec	source
<code>prng:SEED</code>	a specified counter-based generator (SplitMix64 feeding xoshiro256**, fixed in this specification so it can never drift with a dependency), emitting base-2 ³² digits
<code>spigot:C/B</code>	digits of the fractional part of constant $C \in \{\text{pi}, \text{e}, \text{ln2}, \text{liouville}, \text{champernowne}, \text{thue-morse}\}$ in base B, from <code>spigot_stream</code> (feature <code>spigot</code>); the integer part is skipped
<code>human</code>	an interactive chooser (§7.4)
<code>replay:FILE</code>	the choice indices recorded in a trace

7.2 Which source a choice uses

Theorem 8.1(iii) holds *surely* only when each voice has its own source (T2 uses one stream per voice). The player routes choices through *stream labels* (revision 1 called them stream keys; the note now uses “key” for a receipt holding a pitch).

Decision 7.2 (Stream labels). Every receipt occurrence carries a stream label, inherited from the receipt that spawned it. By default the label is the receipt’s subject timbre when that is concrete, and otherwise the digest of its location. It MAY be set explicitly with an annotation for `#A (...)` or on a definition application `Voice(...) #A`; annotations are metadata and change neither the elaborated term’s congruence class nor its digest. The command line maps labels to sources: `--chance prng:7 --stream A=spigot:pi/16 --stream B=prng:12`. A contention set uses the stream of its least receipt occurrence; at a chimeric key that is the first-installed timbre’s stream, which is why a chimera is normally given one explicit label (`Chimera(...) #K`). Sets with a single alternative (every null note) consume nothing.

7.3 Factorised resolution

Lemma 6.4 lets a star whose weights factor as $f(a)g(b)$ be resolved by two independent draws. T4 depends on this: pitch from the hexadecimal digits of π , duration from the base-5 digits of e , one digit each.

Requirement 7.3 (Factor routing). When a star’s clause is a product of factors that are functions of disjoint parts of the view (the elaborator can see this syntactically: a `(pitch, carry)` machine times a `(prev, dur)` machine), the engine MAY resolve it factor by factor, each factor drawing from its own stream (`--stream A.pitch=spigot:pi/16 --stream A.dur=spigot:e/5`). This is an equivalent resolution by Lemma 6.4 and MUST be recorded in the trace. Without a per-factor stream the joint draw is used.

7.4 Human choice

Proposition 7.4 makes the improviser a chance source that ignores the probabilities and chooses among available candidates. Section 9 locates that choice: in the instrument division a player only sends, and choice is made at receipts, so an improvising player needs receipts of its own — listening to a lead sheet, to the band, or to a fan of phrases it offers itself. A human bound to a stream label is choosing at exactly those receipts.

Requirement 7.4 (Human chooser). The `human` source presents, for each set with more than one alternative, the alternatives as notes (timbre, pitch, duration, carried datum), with their normalised probabilities shown for guidance, and accepts an index. Only alternatives of positive weight are offered, so every trace a human produces is admissible, and with a viability factor never runs out (Prop. 7.4(iii)). Mixed performance — a human on one stream label,

algorithmic sources on the others — is the default when `human` is bound to a single label. In this version the human chooser is step-wise, not real time.

8 Outputs

8.1 Playback

Requirement 8.1 (Playback is a function on records). `score-render` MUST expose $\nu : \text{Record} \rightarrow \text{Note}$ and compute every note it prints, writes or sends by applying it to a record. The engine calls the same function to build views. A playback of a run is the sequence of notes of its records; the performance is the multiset of its timed notes of positive length. Given a trace, playback needs neither the score nor the engine (§9, `render`).

8.2 Performance

The performance is printed as a table by default and as JSON with `--out perf.json`:

```
{ "score": "sha256:...", "end": "1",
  "notes": [ { "onset": "0", "timbre": "piano", "pitch": "A3", "dur": "q", "len":
    "1/4" },
    { "onset": "0", "timbre": "piano", "pitch": "A2", "dur": "h", "len":
    "1/2" }, ... ] }
```

Notes are sorted by (onset, timbre, pitch, duration) so that two runs with the same performance produce byte-identical files; this is how interleaving invariance is tested. Null notes are excluded; `--include-null` adds them for diagnosis.

8.3 Trace

Requirement 8.2 (Replayable trace of records). With `--trace run.jsonl` the player writes JSON Lines: a header (format version, player version, score digest, alphabets with lengths and MIDI numbers, timbre declarations, every source spec, stream-label bindings, scheduler, matching mode, locality regime, collection mode), then one event per resolution: step number, onset, location digest, set size, number of alternatives, chosen index, digits consumed per stream, and for each fired candidate its record — both subjects' decorations and the payload decoration, with the payload process by digest. `f1r3score replay` MUST reproduce the event sequence from the score and the trace, and MUST fail if the score's digest differs from the header's.

8.4 MIDI

Requirement 8.3 (Standard MIDI File). `--midi out.mid` writes a format-1 file with one track per rendered timbre, a program change from the timbre's declaration, and note-on/note-off pairs at ticks(t) and ticks($t + |\delta|$). A note's track is chosen by its timbre — the meet — so a chimeric key's notes are routed per note to whichever timbre won. Rests, and every note in `dead` or `ctl`, produce no events. The division (ticks per quarter) is the least value making every onset and length an integer number of ticks, if that is at most 32767; otherwise 960 with rounding and a warning. Tempo is `--bpm` (default 120), a rendering parameter only.

Finding 8.4 (`spigot_midi` cannot be reused as is). `spigot_midi::MidiTrack` writes format 0 from a sequence of (pitch, duration, velocity) notes with no onsets: it is monophonic and sequential by construction. A score is polyphonic, with explicit onsets, simultaneous strikes and, now, per-note timbre. The player needs its own small writer. It SHOULD follow `spigot_midi`'s

dependency-free style, reuse its `GeneralMidi` enumeration for program names, and MAY later be upstreamed to replace it.

8.5 Live MIDI

With feature `live` and `--live [PORT]`, a playback thread receives records from the engine through a bounded channel, applies ν , and sends notes to a MIDI port via `midir`, as `leap_spigot::player` does. The engine runs ahead of the clock by a configurable look-ahead in whole notes and never waits on the clock: live output is a rendering of the same run, and its trace is identical to an offline run's.

9 Command line

```
f1r3score check    piece.score          # parse, elaborate, lint, report
                  fragment and digest
f1r3score expand   piece.score          # print the elaborated core term
f1r3score play     piece.score [options] # run once
f1r3score replay   piece.score run.jsonl # re-run from the trace's choices and
                  compare
f1r3score render   run.jsonl [--midi out.mid] [--out perf.json] # playback only
f1r3score machine  import skein.machine --pitches piece.score    # print as a
                  factor

options for play:
--chance SPEC          default source (prng:SEED | spigot:C/B | human)
--stream LABEL=SPEC    bind a stream label (repeatable; LABEL.pitch / LABEL.dur
                        for factors)
--scheduler S          canonical | random:SEED | by-timbre:A,B,...
--notes N | --until T | --max-steps N
--mode max|one         matching mode (default max)
--gc off|freshness     collection (default off)
--max-alternatives N   bound on maximal matchings per set (default 100000)
--out FILE --trace FILE --midi FILE --bpm N --live [PORT] --include-null
```

`render` is new in revision 2 and exists because reduction is unlabelled: the trace's records already contain everything playback needs (Def. 2.8). Exit codes: 0 success; 1 usage; 2 parse or elaboration error; 3 run-time error (`unproductive`, `freshness-violated`, `alternative limit`); 4 replay mismatch.

10 Conformance

10.1 The ported suite

Each `skeinsim` test becomes a Rust integration test in `score-conformance`, reading its score from a `.score` file (not constructed in Rust), so the suite tests the front end as well as the engine. Statistical thresholds are those the note reports; tests are seeded and deterministic.

test	what	pass criterion
T0	inertness of the unplayed	one live location per step; exactly 79 stale messages per note (16 pitches, 5 durations)
T1	law of one voice (pitch leads)	pitch, duration and joint deviations within sampling error: every $ z \leq 3.5$; no off-machine transition
T2	harmony by independence	five schedulers, byte-identical performances
T3	two-sided fitting	max : first resolution emits $\min(m, n)$ notes; one : 1, then $\min(m, n) - 1$ more
T4	instrument as a score	pitch = hex digits of π , duration = base-5 digits of e , one digit each, 400 notes
T5	law of the dual (rhythm leads)	as T1, via COMM ₂
T6	simultaneity by contention	two notes per resolution; equal durations never
T7	style $\otimes$ physicality	transitions match normalised product; $ z \leq 3.5$; no forbidden transition
T8	idioms from the past	zero triple repeats under the crisp factor; no silence; motif completion within 0.03 of analytic
T9	table = formula	zero difference over 3000 random views (exact rationals), for every table key including dur
T10	reflective = step semantics	identical performance over 3000 notes; exactly two null notes per note, all (r, ϵ) ; every request delivered to the server is a general name, both decorations -
T11	reflective voice, inertness	at most one record location with a candidate; $ \Pi - 1$ dead keys per note
T12	Twinkle	22 notes, end 4, identical under both orders; every written note passes @0
T13	the 4-hands piano (Ex. 9.5, Finding 10.8)	two players: one performance under 20 random schedules, exactly the table of Ex. 9.5, with 16 null notes; with the third player: exactly two distinct performances, the two the example describes
T14	the chimeric instrument (Ex. 9.7, Finding 10.9)	4500 open-timbre touches per condition; vibes share within 0.02 of 0.9, 0.5, 0.1 for eighths, quarters, halves; exactly 1 with vibes written into every touch; no key location ever holds more than two receipts

T10 is the central test of the voice, and T13 of the instrument layer. In both, every component is a score: the reflective **Voice** against an unrolled voice of k written steps; the **Keyboard** and ; of **std.score** against the note’s table. The T14 tolerance allows $|z| \approx 4.7$ at $p = 0.5$ with 4500 touches; the note’s reported shares are within 0.01.

10.2 Additional tests

- **Congruence and round trip.** Normal-form interning agrees with an independent decision procedure on 10^4 random term pairs with wildcards; Proposition 2.5 holds on all of them; `@*x` is never identified with `x` (Rem. 2.6).
- **Matching.** Unit tests for each clause of Defs. 2.4 and 2.7 and Prop. 2.10(ii): open timbre against concrete (matches, note in the concrete timbre); both open (never); open datum on either subject (never); equal polarities (never); payload decoration irrelevant to matching but carried into the bound name.
- **Records.** For every conformance run, the notes the engine reported equal ν applied to the trace’s records, and `render` on the trace produces the same performance and MIDI file as `play`.
- **Cross-implementation differential.** For T0–T14 and random small scores, the Rust player and `skeinsim.py`, driven by the same recorded choice indices, emit the same records. The Python side gains a small adapter that accepts a choice list; this is the faithfulness obligation, kept as a permanent CI job.
- **Replay.** Every conformance run’s trace replays to an identical event sequence.
- **Instruments.** Proposition 9.2: a key location never holds two receipts of one timbre; two touches aimed at one key queue. Proposition 9.4: a synchronous step releases its continuation at $\max_i(t_i + |\delta_i|)$ with two null notes per touch. Proposition 9.6: a chimera’s key location holds exactly one receipt per timbre whenever nothing there is sounding. A hocket between two timbres alternates note by note (§7.4).
- **Enforcement.** A window-behavioural score using `Viable $\beta$` (Prop. 7.2) runs 10^4 notes without violating β and without falling silent; the same score without the viability factor is shown, on a fixture, to fall silent.
- **Chords.** A chord receipt emits k notes at one onset, and its continuation is stamped at the longest duration.
- **Collection.** A deliberately non-fresh score run with `--gc freshness` fails with `freshness-violated`; FourHands run with `--gc freshness` collects nothing and plays as without it (Finding 6.5).
- **Fans over a subset** equal full fans with a crisp exclusion factor, record for record, under the same stream.
- **MIDI.** Twinkle’s and FourHands’ files are byte-identical to golden files; a chimera run routes each note to its winning timbre’s track.

11 Phasing

phase	content	exit criteria
P0	workspace; score-core arena with wildcard decorations, normal forms, matching, instance order, substitution, digests; parser for the core syntax and alphabets; pretty-printer	round-trip parse/print; congruence and round-trip property tests; matching unit tests; Twinkle parses
P1	engine with clause-free receipts: location-only soup, candidates by matching, contention, maximal matchings, time, schedulers; records and playback; performance output; trace; render	T3, T6, T12; records test
P2	local clauses: atoms, spatial formulae, tables (incl. dur), machines, exact weights; PRNG source with interval decoding; macros and par comprehensions	T0, T1, T5, T7, T8, T9
P3	std.score voices: reflective voice, handoff, cross-timbre hocket; stream labels; freshness collection with record-shape test	T2, T10, T11; collection tests
P4	std.score instruments: keys, keyboards, chimeras, touch ; synchronous output ; with fresh acknowledgement locations	T13, T14; instrument tests
P5	MIDI writer; spigot sources and factor routing; replay; live MIDI	T4; replay; MIDI goldens
P6	window-behavioural fragment: $\langle \rangle$, leads , ν ; chord receipts	enforcement and chord tests
P7	human chooser; wasm build of the core crates; cross-implementation CI	differential job green; wasm build in CI

P0–P4 deliver a player for every example in the note except Proposition 7.2 and Definition 6.5; P5 makes it musically usable; P6 completes the calculus as the note defines it. Instruments are a phase of their own because they are where the revised matching rule is exercised hardest: every touch leaves its timbre open, and every key re-arms by reflection.

12 Decisions to confirm

1. **Home of the code.** Candidates: a new repository **F1R3FLY-io/f1r3score**; a directory beside the note in **publications/f1r3score**, where revision 1 of this specification now sits; or **F1R3Games**. Since the calculus is deliberately not F1R3Skein, the proposal is a dedicated repository, with the note’s directory linking to it.
2. **Surface syntax.** Confirm Decision 3.1: a CFL dialect now, restated as **Theory Score** in the DDL later; the name syntax $\langle @P, \tau, d \rangle$ with $_$ for an open component; $@P$ and $x!(Q)$ as the general-name shorthands; the file extension **.score**.
3. **Touch notation.** The note writes $\langle @P, k, d \rangle$ for a touch at key k of keyboard P , which reuses the timbre slot for a key. The proposal is **touch(P, k, d)** in the surface syntax, to keep the triple’s slots meaning one thing each.

4. **Freshness of acknowledgement locations.** Definition 9.3 asks for “a fresh location A ”. A finite term can only make A fresh per elaboration site. That suffices for written phrases, which run once, but a `; inside` code that is re-run by reflection — a player that is itself a self-refreshing server — would reuse A across runs, and acknowledgements from one run could release another. The proposal: fresh per site, with an elaboration error when a `; occurs` under a quote that can be re-run; a per-run fresh A would need the record-of-the-past trick voices use (quote the current location into A), which the note may prefer to specify.
5. **Exact rationals by default.** Decision 7.1 and R5.3 put bit-for-bit reproducibility ahead of speed. The proposal is not to add an `f64` mode until a measured need appears.
6. **Stream labels.** Decision 7.2 defaults to the receipt’s timbre, or its location when the timbre is open, with optional labels. Confirm, and confirm that a chimera’s contention set should draw from one stream rather than, say, the stream of the player whose touch it is.
7. **Scheduler default.** `canonical` makes runs reproducible without a seed. Confirm, or prefer a seeded random default. It matters audibly only where players contend for a key.
8. **Behavioural modalities.** Decision 5.6 evaluates them against the actual company, and R5.5 admits them only in the window fragment. Confirm that a bounded-lookahead approximation, which would not guarantee liveness, should stay out.
9. **Timbre atoms.** No atom reads the note’s timbre or the payload’s in this version, matching Def. 3.1. Confirm, or add `timbre(t)` now as the first step of open thread 4.
10. **Chord continuation.** The longest-duration convention of Def. 6.5 is isolated in one function; confirm it as the default while open thread 5 is pursued.
11. **Handoff record.** Proposition 7.5 notes that the outermost record of a handoff carries a pitch that was never played. The proposal: expose it in `last[...]`, and give `std.score` a `call[...]` atom that skips it.
12. **Pitch naming.** Scientific pitch names with $C4 = 60$, explicit MIDI numbers otherwise, enharmonic spellings distinct data (`F#4` and `Gb4` are different pitches with one MIDI number).